📖 TTSWAP Technical Whitepaper

1 Overview

TTSWAP (Token-Token Swap) is a decentralized market-making protocol (AMM) running on EVM-compatible chains. The protocol matches trades automatically through smart contracts, without relying on any centralized institution. Its core innovation is the **Constant Value AMM (CV-AMM)**: a single set of $O(1)$ algebraic formulas that simultaneously covers Uniswap's CPMM, Balancer's weighted pools, and Curve's StableSwap low-slippage behavior — with **no exponentiation and no numerical iteration**.

TTSWAP's core value propositions:

- **Zero-impermanent-loss LP returns**: divestment returns token amounts based on the historical principal ratio recorded in the proof, independent of the pool's current price; layered on top of a protocol-level triple defense (value anchoring + issuer subordinated buffer + TTS fund).
- **O(1) direct swap between any two tokens**: all goods are attached under the same Singleton contract, and `buyGoodInput → buyGoodOutput` (or `payGoodOutput → payGoodInput`) completes a cross-token swap in two steps; liquidity is aggregated at the protocol layer and no longer fragmented across pairs.
- **Protocol-native six-way fee split**: six parties — LP / Operator / Gate / Referral / Customer / Platform — with contract-level `checkGoodConfig` enforcing that the shares sum to 100%.
- **Singleton + EIP-1153 transient lock + native ETH**: `buyGood` identifies assets uniformly through `T_GoodKey`, and the Relayer mode deducts `executeFee` as compensation.
- **X402 intent-oriented trading**: `buyGood` and `payGood` verify EIP-712 when `msg.sender != _trader`; the Executor pays gas on behalf of the user and receives a fixed `executeFee`.
- **Price-doubling unlock + 100% buyback-and-burn**: TTS token 4C allocation + unlock only when the token price rises, tightly binding unlocking to sell pressure; 100% of community treasury fees are used for secondary-market buyback and burn.

2 Protocol Features

1. **Constant Value AMM**: both ΔV and Δb are solved with algebraic formulas, and the dual coefficient is **hardcoded as the constant 2** in `L_Good.buyGoodInput / buyGoodOutput` (and `payGoodInput / payGoodOutput`), seamlessly switching between CPMM, Weighted, and StableSwap within the same mathematical framework.
2. **Singleton + one-token-one-pool**: `mapping(uint256 goodid => S_GoodState) private goods` + `T_GoodKey` addressing guarantees each asset has exactly one global pool; cross-token swaps route through a value hub, avoiding the N^2 pair fragmentation. The `(ercType, contractAddress, id)` triple of `T_GoodKey` reserves addressing headroom for **tokenized equities (ERC-1155) and prediction-market outcome shares (ERC-6909) and other multi-id assets** (see §4.2.1).
3. **Liquidity amplification (enpower)**: liquidity is amplified by tiering price and contracting the liquidity range.
4. **Zero-IL protocol-level guarantee**: `L_Good.disinvestGood` returns the original amount via `proofInvest × shares / proofShares` plus a liquid-fee share; the triple defense backstops the rest.
5. **EIP-1153 transient storage**: `L_Transient` uses `tstore/tload` to implement the reentrancy lock (`guardedEntry`: `lock 0→2` for a standalone call, keeping the `1→2` nesting level; `lock > 1` reverts with `TTSwapError(3)`) and ETH `msg.value` depth counting; `multicall` sets the lock to 1 via `multicallEntry`, so batched sub-calls share the same ETH budget.
6. **Protocol-native six-way fee split**: `L_Good.allocateFee` automatically distributes the profit from each divestment across six paths — `liquidFee / operatorFee / gateFee / referFee / customerFee / platformFee`.
7. **Native ETH support**: `L_Currency.NATIVE = address(1)`, no WETH wrapping needed, and unused ETH is automatically refunded when the transaction ends.
8. **Proof of Investment + LP mining**: an LP investment automatically triggers `L_Proof.stake(TTS, investor, value)`; a single investment = trading receipt + governance token hashrate + yield receipt.
9. **Price-doubling unlock tokenomics**: 4C allocation (Founder/Partner/Value/Capital 20/12/44/24%) + a per-unlock cap of 1/20, 1/14, 1/12, 1/8 + thresholds judged on-chain by `ishigher`, **fully on-chain**.
10. **X402 trading protocol**: `buyGood` / `payGood` separate signing from execution; the Executor pays gas and deducts `executeFee` from the output token (or the input token on a same-token path).
11. **Upgradeable Proxy deployment**: `TTSwap_Market_Proxy` / `TTSwap_Token_Proxy` hold state, and the `implementation` can be hot-upgraded; the DAO Admin can `disableUpgrade` to permanently lock the upgrade authority.

3 Constant Value AMM Model

3.1 Core Trading Formulas (v2.0.0: `buyGoodInput` / `buyGoodOutput`)

When a user swaps Δa of Token A for Token B, the protocol proceeds in two steps. A single large swap **iterates in slices of 1% of pool depth**, applying the same algebraic kernel to each slice, and validates inventory upper/lower bounds with `safeLine` at the end.

Single-step value mapping (input side, `buyGoodInput`)

$$\Delta V_i = \frac{2 \cdot V \cdot \Delta q_i}{2 \cdot Q + \Delta q_i}$$

Single-step quantity mapping (output side, `buyGoodOutput`)

$$\Delta q_i = \frac{2 \cdot Q \cdot \Delta V_i}{2 \cdot V + \Delta V_i}$$

Corresponding to the inner loop of `L_Good.buyGoodInput` / `buyGoodOutput` (each step processes at most `current_quantity / 100` or `current_value / 100`):

```
// Input side: sell Token A, accumulate transferred value ΔV
value = (2 * _swapParam * current_value)
      / (2 * current_quantity + _swapParam);
swapTemp.get += value;
swapTemp.current_quantity += _swapParam;

// Output side: buy Token B, accumulate output quantity Δq
quantity = (2 * _swapParam * current_quantity)
         / (2 * current_value + _swapParam);
swapTemp.get += quantity;
swapTemp.current_quantity -= quantity;
```

Safe-line constraints:

- Input side: `current_quantity ≤ getSafeLineUpper(currentState.amount0 + goodConfig.amount1())`, otherwise `TTSwapError(55)`;
- Output side: `current_quantity ≥ getSafeLineLower(currentState.amount0 + goodConfig.amount1())`, otherwise `TTSwapError(56)`.

Parameter descriptions (mapped one-to-one to code fields):

Symbol	Meaning	Code field
V	Total value of the Token in the protocol (invested value)	`investState.amount1()`
Q	Current quantity of the Token in the protocol (including accumulated fees + virtual amount)	`currentState.amount1()`
Δa	Net input entering the pool after deducting `sellFee`	`_swapParam` of `buyGoodInput`
ΔV	The "transferred value" produced by this trade (accumulated across slices)	return value `amount1` of `buyGoodInput`
Δb	Gross amount of Token B the user receives (before `buyFee`)	return value `amount1` of `buyGoodOutput`
2	Fixed coefficient in the formula	hardcoded in `L_Good`

> **Design philosophy**: the essence of a swap is a game between the **quantity imbalance a user is willing to bear** and the **protocol's value conservation**; slice iteration + `safeLine` preserve the $O(1)$ per-step algebra while suppressing the instantaneous impact of a large single trade on pool depth.

3.3 Mathematical Equivalence with Mainstream AMMs (formal proofs in Appendices A–D)

- **CPMM (Uniswap V2)**: when $V_A = V_B$, the formula strictly degenerates to $x \cdot y = k$.
- **Weighted Pool (Balancer)**: when $V_A \neq V_B$, the Spot Price is **exactly identical** to Balancer's weighted-pool $SP = \frac{Q_A W_B}{Q_B W_A}$ (mapping V to W), but **requires no exponentiation**.

3.4 Path Independence / Closure

Rigorous algebraic derivation of the code logic satisfies:

- **Reverse-swap closure** (Appendix I): after $A \to B$ then $B \to A$, the state is strictly restored in the fee-free case.
- **Split-trade error** (Appendix J): the output difference between a single Δa and splitting it into $x_1 + x_2$ is $O\left(\left(\frac{\Delta a}{Q_A}\right)^2 \cdot \frac{V_A^2 - V_B^2}{V_B^2}\right)$, negligible in engineering practice.
- **Triangular closure** (Appendix K): $A \to B \to C \to A$ strictly returns to the initial state.

3.5 Post-Trade State Update (`buyGood` write-back order)

`buyGood` call chain: `good1change = g1.buyGoodInput(amount0)` → `good2change = g2.buyGoodOutput(good1change.amount1())` .

Side	`currentState.amount0` (actual pool inflow)	`currentState.amount1` (current quantity)	`investState`
Token A (input)	+ `sellFee`	+ `swapParam` (net input + fee)	unchanged
Token B (output)	+ `buyFee`	− `outputGross` (pool-side change before fee deduction)	unchanged

After each state-change path ends, the relevant good calls `goodConfig.updateRunBlockConfig()` to refresh the **run-block slot** (`block.number % 4095` , bits 196–185). `investGood` updates the invested good; `buyGood` updates **only the output-side** good2; `payGood` updates good2 at the start of the flow. A second write to the same good within the same block slot reverts with `TTSwapError(46)` . `investState` only changes on invest / divest / welfare injection.

3.6 Run-Block Anti-Replay (`updateRunBlockConfig`)

v2.0.0 uses a **block slot** (not a timestamp) to limit repeated state writes to a single pool within a single block:

```
// L_GoodConfig.updateRunBlockConfig
uint256 slot = block.number % 4095;
if (config.getRunBlockConfig() == slot) revert TTSwapError(46);
return (config & ~mask) | (slot << 185);
```

`_checkGoodActive` also validates `getRunBlockConfig() == block.number % 4095` at entry, blocking a busy slot early.

Entry	good whose run-block is updated	Integration note
`investGood`	invested good	cannot invest the same good consecutively within the same block
`buyGood`	**only good2 (output)**	a second buy of the same output token in the same block fails; the input token is unaffected by this write
`payGood`	**good2 (output)**, written before AMM computation	a second pay of the same output token in the same block fails

Aggregators / test harnesses should `vm.roll` to increment `block.number` between consecutive operations, or use `multicall` to orchestrate multiple steps into the same transaction while avoiding a repeated slot on the same good.

4 Token Model

4.1 The Six-Dimensional State of a Token

Property	Meaning	Code field
Market Value V	in-pool weight / invested value	`investState.amount1()`
Current Quantity Q	current quantity (including fees + virtual amount)	`currentState.amount1()`
Actual Pool Inflow I	cumulative actual tokens entering the pool	`currentState.amount0()`

Property	Meaning	Code field
Shares S	total LP shares	`investState.amount0()`
Virtual Amount	leveraged virtual depth	`goodConfig.amount1()` (low 128 bits)
Config	fee rate / fee split / flags	`goodConfig` (high 128 bits)

4.2 `T_GoodKey` and Good ID

In v2.0.0 all on-chain entries identify assets by `T_GoodKey`, no longer passing `address` directly:

```
struct T_GoodKey {
    uint8 ercType;          // 1: ERC20, 2: ERC1155, 3: ERC6909 (currently only ERC20 is enabled)
    address contractAddress;
    uint256 id;             // fixed to 0 for ERC20
}
```

`goodid = T_GoodKey.toId()`: ERC20 / native ETH takes `uint160(contractAddress)`; ERC-1155 / ERC-6909 takes `keccak256(packed, id)` (where `packed = (ercType << 160) | uint160(contractAddress)`). The `goods` mapping is keyed by `uint256 goodid`.

> **Current status**: v2.0.0 enables only `ercType == 1` (ERC20 / native ETH); the `toId` / `transferFrom` branches for `ercType == 2/3` are retained in `T_GoodKey.sol` but currently `revert TTSwapError(42)`.

4.2.1 Why the `(ercType, contractAddress, id)` Triple — An Addressing Layer for Asset Expansion

`T_GoodKey` decouples "asset standard + contract + sub-id", which is the protocol's core design point for **future asset-class expansion**. The `id` field is always 0 for ERC20, but for multi-id asset standards (ERC-1155 / ERC-6909) it distinguishes **different underlyings under the same contract**. This precisely fits two classes of "one-contract-multi-id" assets:

Expansion direction	Natural mapping	`T_GoodKey` encoding	One-token-one-pool effect
Tokenized Equity / RWA	one issuing contract custodies multiple stocks, `id` = individual stock identifier	`ercType=2` (ERC-1155), `contractAddress=issuing contract`, `id=stock number`	each stock occupies its own shelf under the Singleton, sharing the same checkout with existing USDT / ETH shelves
Prediction Market Contract	each outcome (YES / NO / multi-outcome) under a market contract is an outcome share	`ercType=3` (ERC-6909), `contractAddress=market contract`, `id=outcome number`	each outcome share is an independent good, directly swappable with any value Token via `buyGood`

Design implications:

- **Onboarding new asset classes with zero core changes**: CV-AMM, `safeLine`, six-way fee split, and `disinvestProof` all operate on `uint256 goodid` as the unit of operation, **independent** of whether the underlying is ERC20 or ERC1155/6909. A new asset class only needs to open the corresponding `ercType` branch in `T_GoodKey.toId()` / `transferFrom()` to be mounted; the business logic layer requires no rewrite.
- **`goodConfig.getContractType` coordination**: the `contractType` field (`L_GoodConfig.getContractType`) can annotate the asset class at the config layer (normal token / equity / prediction outcome, etc.), enabling frontends and risk control to display and cap by class without polluting `goodid` addressing.
- **Cross-underlying O(1) direct swap unchanged**: once tokenized equities or prediction shares are listed, they can be swapped with value Tokens such as USDT / ETH in two steps via `buyGoodInput → buyGoodOutput`, without building a separate pair for each new underlying — exactly the natural fit of the Singleton one-token-one-pool for RWA / prediction markets.

4.3 Token Classification

Class	Flag	Description
Value Token (Value)	`isvaluegood() = true` (bit 255)	protocol-anchored assets (USDT / ETH / WBTC, etc.). After `initGood`, the **Market Admin** sets the `isValueGood` bit via `modifyGoodByAdmin`.

Class	Flag	Description
Normal Token (Normal)	`isvaluegood() = false`	long-tail assets. Trading is bounded by the `safeLine` upper/lower limits (`TTSwapError(55/56)`), preventing pool depth from being drained instantaneously.

> All goods launch as a single token through the unified `initGood`; the initial config is fixed to `L_GoodConfigLibrary.setInitialConfig()`, with default fee rates and fee splits in Appendix E.

4.4 Token Config: 256-bit Bit-Packed Storage

`goodConfig` is a `uint256`: the high 128 bits are static/semi-static config, and the low 128 bits (`amount1`) record the runtime virtual-amount increment.

Three-tier modification permissions (v2.0.0)

Section	Bit range	Who can modify	Typical fields
Super Admin section	255–247	Market Admin (`modifyGoodByAdmin`)	`isValueGood`, reserved ERC type
Admin section	246–191	Market Manager (`modifyGoodByManager`)	`isFreeze`, `isVerified`, `isPromise`, six-way fee split, `limitPower`, `safeLine`
User section	166–128	Good Owner (`modifyGoodByGoodOwner`)	`power`, trading fee rates, `disinvestChips`

Market-adjustable section (Manager writes, must pass `checkGoodConfig`)

Bit	Config	Bits	Accessor	Unit	Default
255	Value Token flag	1	`isvaluegood()`	bool	0
246	Freeze flag	1	`isFreeze()`	bool	0
245	Verified flag	1	`isVerified()`	bool	0
244	Promise-application flag	1	`isPromised()`	bool	0
243–241	LP fee split	3	`getLiquidFee()`	`× 0.1`	6
240–237	Operator fee split	4	`getOperatorFee()`	`× 0.02`	1
236–234	Gate fee split	3	`getGateFee()`	`× 0.04`	5
233–229	Referral fee split	5	`getReferFee()`	`× 0.01`	8
228–224	Customer fee split	5	`getCustomerFee()`	`× 0.01`	8
223–219	Platform fee split	5	`getPlatformFee128()`	`× 0.01`	2
218–214	Max amplification	5	`getLimitPower()`	`× 100`	1
213–204	Safe line	10	`getSafeLineUpper/Lower()`	raw	80

> The sum of the six fee-split items is enforced to equal 100% by `checkGoodConfig()`.

User-adjustable section (Good Owner modifies via `modifyGoodByGoodOwner`)

Bit	Config	Bits	Accessor	Unit	Default
166–162	Amplification power	5	`getPower()`	`× 100`	1
161–154	Disinvest chips	8	`getDisinvestChips()`	`amount/storedValue×4`	10
153–148	Invest fee rate	6	`getInvestFee()`	1/10000	8
147–142	Disinvest fee rate	6	`getDisinvestFee()`	1/10000	8

Bit	Config	Bits	Accessor	Unit	Default
141–135	Buy fee rate	7	`getBuyFee()`	1/10000	8
134–128	Sell fee rate	7	`getSellFee()`	1/10000	8

4.5 Singleton + One-Token-One-Pool: The Supermarket-Shelf Intuition

TTSWAP attaches every Token's pool under the same `TTSwap_Market` contract's `mapping(uint256 goodid => S_GoodState) private goods`, addressed by `T_GoodKey`. **Each asset corresponds to one shelf, and the entire supermarket has only one checkout.**

Traditional DEX: One "Specialty Store" per Pair of Goods

Take Uniswap V2: to swap "bread" for "milk", someone must first open a "bread ↔ milk" specialty store (pair). If none exists, you can only do "bread → cash (ETH/USDC) → milk" — walking into two different stores and paying the toll twice.

Supermarket-world analogy: **every pair of goods needs its own exchange window.**

Number of goods	Exchange windows (pairs) needed
3 kinds (bread, milk, eggs)	3
10 kinds	45
100 kinds	4,950
N kinds	$\frac{N(N-1)}{2}$

Problems:

- Each window needs **independent stock** (two-sided liquidity); the bread and milk inventory in the "bread ↔ milk" window is **completely disconnected** from the inventory in the "bread ↔ eggs" window;
- Newly listed long-tail goods usually have no window opened for them, meaning they **cannot be traded at all**;
- Users must learn routing: which store to visit, how many stores to hop through, how many slippage events, how many gas payments.

TTSWAP: One Shelf, Any Swap

TTSWAP puts all goods on the **same shelf** of the same supermarket. Each good has its own slot (the V/Q quadruple), but all slots share the same **checkout** — the protocol's internal "value hub".

The bread-for-milk swap process (corresponding to `buyGood`'s `buyGoodInput → buyGoodOutput` two steps):

1. **The checkout prices the bread first**: it weighs how much bread you have, how much bread is on the shelf right now (Q_A), and how much total value this batch of bread corresponds to (V_A), computing the "purchasing power" ΔV the bread brings;
2. **Then it uses that purchasing power to exchange for milk**: it checks how many bottles of milk are on the milk shelf (Q_B) and how much total value they correspond to (V_B), and computes how much milk Δb you can take away.

The entire process completes in **a single on-chain call**; you never leave this supermarket and never pass through any intermediate good.

Two Intuitive Benefits of the Same Shelf

1. **O(1) direct swap between any two goods**
   Whether it's "bread for milk" or "eggs for soybeans", it's the same two-step operation: $\text{good}_A \to \Delta V \to \text{good}_B$. No need for multi-hop routing like `bread → cash → milk`.
   The $\frac{N(N-1)}{2}$ swap paths among N goods **share the same checkout in TTSWAP**.
2. **Liquidity aggregation**
   When the supermarket receives a new batch of bread, that bread's liquidity can be shared by "all bread-related swaps", not just the single "bread ↔ milk" window. This is the true meaning of §1's "**O(1) direct swap between any two tokens**" and "**liquidity aggregated at the protocol layer, no longer fragmented across pairs**".

Comparison with a Traditional Supermarket

Dimension	Traditional DEX (one store per pair)	TTSWAP (one supermarket)
Swap paths between goods	N goods need $\frac{N(N-1)}{2}$ pairs	each good needs only one shelf
Listing a new good	needs extra capital to build a pair; long-tail goods get none	just place it on a shelf to be swappable by everyone
Liquidity utilization	fragmented across each pair, each fighting alone	all shelves share the same checkout's liquidity depth
User routing complexity	must find a path (single-hop / multi-hop / aggregator)	always "two steps"
Gas cost	multiple cross-contract calls	a single call, completed within the Singleton

In one sentence: the Solidity structure `mapping(uint256 goodid => S_GoodState) private goods` + `T_GoodKey.toId()` compresses DeFi's "N specialty stores" into TTSWAP's "one large hypermarket" — **goods no longer need to be paired two-by-two, and any good can be swapped for any other good**.

5 Liquidity Management: Invest and Divest

5.1 Proof of Investment

Each time an LP provides liquidity, the protocol records in `mapping(uint256 proofid => S_ProofState) private proofs`:

```
struct S_ProofState {
    uint256 currentgood;   // invested target goodid
    uint256 shares;        // amount0: shares
    uint256 state;         // amount0: virtual value, amount1: actual value
    uint256 invest;        // amount0: virtual quantity, amount1: actual quantity
}
```

`proofid = keccak256(abi.encode(owner, currentgood))`.

5.2 Investment Path (v2.0.0 single-token entry)

Function	Use case	Permission	Input
`initGood`	launch a new single-token good with a custom initial price	any user	`T_GoodKey` + `_initial` (amount0=value, amount1=quantity)
`investGood`	append single-token liquidity to an existing pool	any user	`T_GoodKey` + `_invest.amount1` (token quantity)

`initGood` constraints:

- `_initial.amount1` $\in$ [500,000, 2^{109}] (`TTSwapError(36)`);
- `_initial.amount0` $\in$ [5×10^{14}, 2^{109}] (`TTSwapError(35)`);
- the initial `goodConfig` is **fixed** to `setInitialConfig()`, not customizable at init;
- the caller automatically becomes the `owner` and creates the first proof;
- **`initGood` does not call `L_Proof.stake`**: the first liquidity only mints proof shares, and TTS hashrate begins accruing only on a later `investGood` (and when the good is marked `isPromised`).

`investGood` core flow:

1. `enpower = goods[goodid].getInvestPower()` (dynamically scaled by the in-pool Q/I ratio, capped by `getPower()`);
2. transfer in `_invest.amount1` tokens;
3. `L_Good.investGood(quantity, result, enpower)`:
   - `investFeeQuantity = getInvestFee(quantity)`;
   - `investQuantity = (quantity − fee) × enpower / 100` (virtual amplification);
   - `investValue = poolPrice × investQuantity`, truncated by `getInvestThreshold`;
   - `investShare` minted proportionally to shares;

4. write back to the pool: `currentState` , `investState` , `goodConfig.amount1` (virtual amount);
5. update the proof; if `goodConfig.isPromised()` is true, then `L_Proof.stake(TTS, msg.sender, investvalue)` and record it proportionally into the proof's `shares.amount1` ; otherwise the stake amount is 0.

> v2.0.0 removed the owner self-quote branch (`isInvestBlocked` / `_invest.amount0 > 0`); appended liquidity is always priced automatically at the in-pool current price.

5.3 Divestment Path

`disinvestProof(_proofid, _goodshares, _gate, _trader, signature)` performs the following chained operations:

1. **Ownership check**: `S_ProofKey(_trader, currentgood).toId() == _proofid` ;
2. **Promise lock**: if `isPromised() && owner == _trader` , revert `TTSwapError(40)` (the owner cannot divest during the initial promise period);
3. **Call `L_Good.disinvestGood`** :
   - `actualDisinvestQuantity = proofInvest1 × _goodshares / proofShares0` (**returned by the proof's historical ratio**);
   - compute profit + `getDisinvestFee` + `allocateFee` six-way distribution;
4. **Transfer**: `toGoodKey().safeTransfer(msg.sender, commission - 1)` sends the accumulated commission + principal to the LP;
5. **Sync TTS unstake**: call `L_Proof.unstake(TTS, msg.sender, disinvestTTSValue)` only when `disinvestTTSValue > 0` (divestment can skip unstake when `initGood` did not stake).

> The `signature` parameter is retained for ABI compatibility and is **not verified**; `_trader` must equal `msg.sender` .

5.4 The Triple Defense Against Impermanent Loss

TTSWAP's LPs reclaim tokens by **historical principal quantity** on divestment, independent of the pool's current price, effectively externalizing IL into "last-exit-taker risk". The protocol converges this risk through a triple defense:

1. **Protocol-level value anchoring**: the `disinvestGood` code path computes strictly by proof history, giving a formal guarantee on the LP's principal quantity.
2. **Issuer subordinated buffer**: the initial liquidity of a meta/value good is injected by the issuer, effectively constituting "subordinated principal" that absorbs volatility first under extreme conditions.
3. **TTS governance-token compensation fund**: extreme shortfalls are backstopped by TTS auction / release, with the DAO Admin's `mint/burn` authority as the last resort (Whitepaper §12.7).

5.5 Disinvest Chips

`getDisinvestChips(investState.amount1()) ≥ disinvestvalue.amount0()` limits the value ratio of a single divestment (configured by the Token Owner). A single divestment exceeding the cap reverts with `TTSwapError(26)` or `TTSwapError(27)` .

5.6 Initial Promise Mechanism (Promise + `refreshPromise`)

The Good Owner can set the `isPromised` flag (bit 244) in `goodConfig` , used together with `refreshPromise(proofid)` :

- the caller **must be the proof owner themselves**: `S_ProofKey(msg.sender, currentgood).toId() == proofid` ;
- if `goodConfig.isPromised() && owner == msg.sender` , the protocol broadcasts the `e_getPromiseProof(currentgood, proofid)` event, consumable by off-chain Launchpad / IDO systems;
- `refreshPromise` **does not support relayers and does not accept EIP-712 signatures**;
- during the promise period, the owner's call to `disinvestProof` is blocked by `TTSwapError(40)` .

5.7 Contract Parameters and Limits (measured values)

Parameter	Limit	Source location	Meaning
Initial quantity lower bound	500,000	`initGood._initial.amount1() < 500000`	`TTSwapError(36)`
Initial quantity upper bound	2^{109}	`initGood._initial.amount1() > 2**109`	`TTSwapError(36)`
Initial custom value lower bound	5×10^{14}	`initGood._initial.amount0() < 5e14`	`TTSwapError(35)`

Parameter	Limit	Source location	Meaning
Initial custom value upper bound	2^{109}	`initGood._initial.amount0() > 2**109`	`TTSwapError(35)`
Minimum append value	10^{12}	`investValue < 1e12` inside `investGood`	`TTSwapError(38)`
Minimum trade value	10^{8}	`buyGood` checks `good1change.amount1() < 1e8`	`TTSwapError(14)`
Quantity overflow protection	2^{109}	`currentState.amount1() + delta > 2**109`	`TTSwapError(18)`
Executor compensation	**50,000,000,000** = 5×10^{10}	`executeFee` constant	at 12-decimal stablecoin precision = **0.05 USDT**

5.8 Safe-Line Constraints

`safeLine` defines the tradeable range of pool depth:

- **Sell-in side** (`buyGoodInput` / `payGoodInput`): `Q` must not exceed `getSafeLineUpper(I + virtual)` , otherwise `TTSwapError(55)` ;
- **Buy-out side** (`buyGoodOutput` / `payGoodOutput`): `Q` must not fall below `getSafeLineLower(I + virtual)` , otherwise `TTSwapError(56)` .

`safeLine` is implemented in the code as **two independent 8-bit fields**: `getSafeLineUpper` (bits 219–212) and `getSafeLineLower` (bits 211–204). **The default stored value is 0**, in which case the upper/lower bounds fall back directly to the base inventory `I + virtual` (no extra constraint); for a non-zero stored value v, the bound $= v \times (I + \text{virtual})/100$, configured by the Market Manager via `modifyGoodByManager` .

```
// before buyGoodInput / payGoodInput ends
if (current_quantity > goodConfig.getSafeLineUpper(
        currentState.amount0() + goodConfig.amount1()
    )) revert TTSwapError(55);

// before buyGoodOutput / payGoodOutput ends
if (current_quantity < goodConfig.getSafeLineLower(
        currentState.amount0() + goodConfig.amount1()
    )) revert TTSwapError(56);
```

Semantics: **a normal token's sell/buy freedom is bounded by the pool-depth safe range**, preventing a large single trade from instantaneously pushing the pool into an unsustainable state.

5.9 Independent Liquidity Definition

- any Token launches as a **single token** via `initGood` , with no need to pair a value Token;
- append single-sided liquidity via `investGood` ;
- divestment settles independently by this proof's shares;
- price is determined by the pool's own $(V, Q, I, power)$;
- `safeLine` takes effect independently per pool.

5.10 Unified Price Constraints

(A) Investment: v2 always prices appended liquidity automatically at the in-pool price (`investGood` has no self-quote branch).

(B) Trade slippage:

- `buyGood` exact-in: `good2change.amount1() >= _swapQuantity.amount1()` , otherwise `TTSwapError(15)` .

(C) Pool-depth health: the `safeLine` upper/lower bounds (see §5.8).

5.11 Single-Sided Liquidity Feasibility: How the Safe Line Prevents Pool Drainage

> **Core proposition**: v2 uses `safeLine` to define each pool's tradeable depth range; large sell/buy pressure reverts when it hits the boundary, forcing the market to complete price discovery through multiple small orders or reverse arbitrage.

5.11.1 Safe-Line Constraint

For all goods (both value Tokens and normal Tokens), the swap path validates after the slice loops in `buyGoodInput` / `buyGoodOutput` (and the pay reverse path) end:

$$Q_{\text{lower}} \le Q \le Q_{\text{upper}}$$

where the upper/lower bounds are computed by `getSafeLineLower/Upper(currentState.amount0 + goodConfig.amount1())`. Crossing either bound reverts with `TTSwapError(56)` / `TTSwapError(55)`.

5.11.2 The Attack Path Is Blocked

When an attacker continuously sells the normal token `$JUNK` to extract USDT:

1. each sell order pushes up $JUNK$'s Q;
2. when $Q > Q_{\text{upper}}$, `buyGoodInput` reverts with `TTSwapError(55)`;
3. selling can only continue after buy-side flow pulls Q back or an LP divests to release room.

Value Token vs Normal Token: both share the same `safeLine` mechanism; the difference is that a value Token is usually elevated by Admin (`isValueGood`), with deeper pools and more abundant liquidity.

5.11.3 The Everyday Supermarket-Restocking Analogy

> A supplier delivered **1,000 bottles of mineral water** to a community supermarket at a wholesale price of **10 yuan/bottle**.
> The supermarket's shelf (= pool Q) can hold at most these 1,000 bottles; if nobody buys but the supplier wants to deliver another 500, the owner will surely refuse — **because the shelf can't hold them; the water would just rot in the warehouse and crowd out other goods.**
> Only when the market actually starts buying water (200 bottles of space open up on the shelf) can the supplier restock another 200. **The buyer must act first before the seller has room to "keep selling".**

TTSWAP handles normal tokens exactly isomorphically to a supermarket:

Supermarket world	TTSWAP counterpart
Shelf capacity	$I + \text{virtual quantity}$ (the inventory ceiling supported by LPs)
Supplier restocking	user selling a normal token (`buyGood` A → B, A being the normal token)
Consumer buying water	user buying a normal token (`buyGood` B → A)
Owner refusing excess delivery	`TTSwapError(55)` revert
Milk and biscuits on other shelves	other value assets in the protocol (USDT / ETH …)

If the owner accepted anything, the supplier would keep dumping goods → the supermarket's cash, shelves, and other goods would all be crowded out by "rotting water" — the classic "normal-token reverse drainage" attack in DeFi. TTSWAP uses `safeLine` upper/lower bounds to **shut down this path directly at the contract layer**.

5.11.4 Value Token vs Normal Token

Dimension	Value Good	Normal Good
Definition	`isvaluegood() == true`	`isvaluegood() == false`
Typical examples	USDT / ETH / WBTC	new project tokens, long-tail assets
Admin elevation	`modifyGoodByAdmin` sets bit 255	not set by default
Liquidity append	`investGood` priced linearly at in-pool price	same as left, shares `L_Good.investGood`
Trade-depth constraint	`safeLine` upper/lower bounds	same as left

v2.0.0's investment path no longer distinguishes value/normal branch formulas; it uniformly uses `L_Good.investGood + getInvestThreshold`.

5.11.5 The Price-Repair Loop on External Price Crashes

> Extreme scenario: a normal token's price on a CEX / other DEX crashes 50%, and arbitrageurs rush to TTSWAP to sell.

The protocol responds in the following order, forming a **gradually converging, non-collapsing** loop:

1. **Sell pressure rises → safe line approaches**: batched sell orders fill, and Q quickly approaches Q_{upper}.
2. **Cap reached → protocol stops further fills**: the next `buyGood` A→B is blocked by `TTSwapError(55)` .
3. **External arbitrageurs step in reverse**: as the spread widens, arbitrageurs buy in the opposite direction, pulling the in-pool price back toward the real price.
4. **LPs divest to release room**: `disinvestProof` returns by proof historical quantity, decoupled from the current pool price (§5.4).
5. **Owner appends liquidity**: via `investGood` at the in-pool current price, freeing depth for the next round of trading.

These 5 steps essentially amortize an "external crash" into many small price-discovery steps, rather than draining the protocol instantly. **The LP's loss comes from time, not from instantaneous slippage amplification.**

5.11.6 Why This Design Is Called a "Safety Boundary", Not "Trade Limiting"

- **swaps between value tokens** (e.g. USDT ↔ ETH) and **normal-token buys** proceed normally;
- **large sell-ins/buy-outs** are only limited as they approach the `safeLine` boundary;
- users can at any time actively adjust pool depth via **divestment** (`disinvestProof`) or **welfare injection** (`goodWelfare`).

Therefore `safeLine` is a **seatbelt that prevents an external price collapse from penetrating into the protocol's balance sheet**, not a throttle on normal trading.

5.11.7 Final Summary

TTSWAP's single-sided liquidity is feasible thanks to four code-layer mechanisms:

1. **`safeLine` upper/lower bounds** (`TTSwapError(55/56)`): limit the impact of single and cumulative swaps on pool depth;
2. **Slice-iterated swaps** (`buyGoodInput/Output` , `payGoodInput/Output`): large trades are priced in steps of 1% depth;
3. **Proof historical accounting + triple defense** (§5.4): the LP's divestment return quantity is decoupled from the current pool price;
4. **Dynamic `getInvestPower`** : scales the leverage of appended liquidity by the Q/I ratio.

5.12 Security Attack Surface (code-layer defense mapping)

Attack	Defense mechanism	Code location
Low-liquidity price manipulation	minimum initial value / initial value lower bound	`initGood`
Sandwich	EIP-712 `external_info` deadline + `minOut`	`buyGood`
Inventory drainage / over-buy	`safeLine` upper/lower bounds `TTSwapError(55/56)`	`buyGoodInput` / `buyGoodOutput` / `payGoodInput` / `payGoodOutput`
Reentrancy	EIP-1153 transient lock `guardedEntry` + `multicallEntry`	`L_Transient`
Same-block double-write	run-block slot `TTSwapError(46)`	`updateRunBlockConfig` / `_checkGoodActive`
Double commission	`referral = gate ? 0 : referral` + `isBan()` zeroing	`disinvestProof`
Welfare over-injection	`currentState.amount0() + welfare > 2**109` revert	`goodWelfare`
Pool-creation front-run	`if (goods[goodid].owner != 0) revert TTSwapError(5)`	`initGood`
Divesting during the promise period	`isPromised && owner` block	`disinvestProof`

6 Trading Functions

TTSWAP v2.0.0 provides three trading entry categories: **`buyGood` (Exact-In)**, **`payGood` (Exact-Out / same-token transfer)**, and **`multicall` (batch orchestration)**.

6.1 `buyGood` (Exact-In Buy)

```
function buyGood(
    T_GoodKey memory _goodKey1,   // token to sell (input good)
    T_GoodKey memory _goodKey2,   // token to buy (output good)
    uint256 _swapQuantity,        // amount0: input amount, amount1: minimum gross output (slippage floor)
    address _recipient,           // referrer: binds referral when non-zero and ≠ trader
    bytes calldata data,
    address _trader,
    bytes calldata signature,
    uint256 external_info         // low 64 bits: deadline (non-zero and expired → TTSwapError(49))
) external payable returns (uint256 good1change, uint256 good2change);
```

Behavior summary:

- AMM path: `g1.buyGoodInput(amount0)` → `g2.buyGoodOutput(ΔV)` ;
- when `_goodKey1.toId() == _goodKey2.toId()` , revert `TTSwapError(9)` (**no same-token fast path**; use `payGood` for same-token payment);
- `good1change.amount1() < 1e8` → `TTSwapError(14)` (minimum transfer value);
- `good2change.amount1() < _swapQuantity.amount1()` and `amount1 > 0` → `TTSwapError(15)` (slippage);
- Relayer: when `msg.sender != _trader` , verify EIP-712; deduct `executeFee` from the **output token** and record it into `g2.commission[msg.sender]` ;
- after finishing, **only g2** calls `updateRunBlockConfig()` .

Event: `e_buyGood(sellgood, forgood, swapvalue, good1change, good2change, trader, external_info)` .

6.2 `payGood` (Exact-Out Payment / Same-Token Fast Path)

```
function payGood(
    T_GoodKey memory _goodKey1,
    T_GoodKey memory _goodKey2,
    uint256 _swapQuantity,   // cross-token: amount0=max input, amount1=target gross output
                             // same-token: amount1=transfer amount
    address _recipient,      // required (address(0) → TTSwapError(32))
    bytes calldata data,
    address _trader,
    bytes calldata signature,
    uint256 external_info    // low 64 bits: deadline (expired → TTSwapError(53))
) external payable returns (uint256 good1change, uint256 good2change);
```

Cross-token path (`_goodKey1 != _goodKey2`):

1. **first** execute `updateRunBlockConfig()` on good2;
2. `good2change = g2.payGoodOutput(amount1)` → `good1change = g1.payGoodInput(ΔV)` ;
3. the actual input `good1change.amount1() + good1change.amount0()` must not exceed `_swapQuantity.amount0()` (max in);
4. Relayer deducts `executeFee` from the output token (same pricing method as `buyGood`).

Same-token fast path (`_goodKey1 == _goodKey2`):

- bypasses the AMM, directly `transferFrom` → `safeTransfer` sends `_swapQuantity.amount1` to `_recipient` ;
- in Relayer mode, deducts `executeFee` from the input token and records it into `commission[msg.sender]` ;
- suited for same-token scenarios like X402 merchant collection and payroll.

Event: `e_payGood(...)` (cross-token and same-token field encodings differ slightly; see `I_TTSwap_Market.sol`).

6.3 `multicall` (Batch Call)

```
function multicall(bytes[] calldata data)
    external payable returns (bytes[] memory results);
```

- implements `IMulticall_v4` , batching sub-calls via `delegatecall` ;
- modifier chain: `msgValue` + `multicallEntry` (lock level 1) + sub-function `guardedEntry` (1→2);

- the outer `msg.value` initializes the ETH budget only once across the whole batch (`L_Transient`), ideal for orchestrations like "approve + swap + collect commission";
- note the run-block constraint: the same good still cannot be written repeatedly within the same block slot, so the batch path must plan the good order or execute across blocks.

6.4 X402 Trading Protocol (Separation of Signing and Execution)

EIP-712 type hashes (Market `Version = "2.0.0"`):

buyGood:

```
buyGood(address _trader,address referral,uint256 _goodid1,uint256 _goodid2,
        uint256 _swapQuantity,bytes data,uint256 external_info,uint256 nonce)
```

payGood:

```
payGood(address _trader,address recipient,uint256 _goodid1,uint256 _goodid2,
        uint256 _swapQuantity,uint256 external_info,bytes data,uint256 nonce)
```

- `nonces[_trader]` increments in the signature-verification path; users can call `cancelNonce()` to invalidate unused signatures;
- Executor commissions are collected in batch via `collectCommission` ; pricing follows the `executeFee = 5×10^10` constant + conversion at the output good's current price.

6.5 Meta-Transaction Coverage

Function	Relayer / EIP-712	Notes
`buyGood`	✅	full signature verification
`payGood`	✅	full signature verification
`investGood` / `disinvestProof` / `initGood` , etc.	❌	`signature` parameter retained for ABI compatibility, **not verified**; `_trader` must equal `msg.sender`
`refreshPromise`	❌	only the proof owner can call, no meta-tx

7 Fees and Commission Allocation

7.1 Fee Sources

Every swap / invest / disinvest / welfare accumulates fees in the pool:

- **Trading**: `buyFee / sellFee` accumulate via the `swap_fee` path into `currentState.amount0` and `currentState.amount1` ;
- **Investment**: `investFee` is deducted from the virtual quantity, and the actual quantity after deduction is added to `currentState` ;
- **Divestment**: `disinvestFee` is proportionally rebated from LP profit back into `currentState.amount0` (in-pool compounding).

LP net value increases monotonically with in-pool fee accumulation; the `safeLine` constraint ensures the net value cannot be instantaneously drained by arbitrageurs.

7.2 `allocateFee` Six-Way Distribution (Protocol-Native)

On divestment, `L_Good.allocateFee` streams the profit in the following order:

1. **Platform**: `platformFee = getPlatformFee128(profit)` , allocated to `commission[address(0)]` (protocol treasury);
2. the remaining `profit -= marketfee` is used as the base, then further split:
   - **LP**: `liquidFee = getLiquidFee(profit)` → `commission[sender]` ;
   - **Operator**: `sellerFee = getOperatorFee(profit)` → `commission[owner]` ;

- **Gate**: `gaterFee = getGateFee(profit)` → `commission[gate]`;
- **Referral**: `referFee = getReferFee(profit)` → `commission[referral]`;
- **Customer**: `customerFee = getCustomerFee(profit)` → sender or gate (depending on the referral path);

The distribution logic has two branches:

Scenario	LP receives	Operator	Gate	Referral	Customer	P
No referral	`liquidFee + principal`	—	`operatorFee + customerFee`	—	—	`remaining p`
With referral	`liquidFee + customerFee + principal`	if owner exists → `operatorFee`	if gate exists → `gaterFee`	`referFee`	—	remainder

> **Config constraint**: `checkGoodConfig` enforces that the six shares (liquid / operator / gate / referral / customer / platform) sum to 100%.

7.3 Commission Collection

- commissions accumulate in the `S_GoodState.commission` mapping;
- the code reserves a **1-unit sentinel** (`commission[addr] = 1`) to avoid a cold SSTORE, so the actual withdrawable amount = `commission - 1`;
- `collectCommission(address[] goodids, ...)` handles at most 100 goods per call;
- when the Market Admin calls `collectCommission`, it batch-clears `commission[address(0)]` (the protocol treasury) — this is **the only on-chain outlet for protocol-treasury reflow**.

7.4 `goodWelfare` Welfare Injection

The issuer can inject tokens directly into the pool as welfare:

```
goods[goodid].currentState = add(
    goods[goodid].currentState,
    toTTSwapUINT256(welfare, welfare)
);
```

This increases both `currentState.amount0` and `amount1`, **issues no new shares**, and directly raises the LP net value. It is bounded by the `currentState.amount0() + welfare > 2**109` overflow protection.

7.5 Initial Operating Rights and Operating-Rights Auction (Operator Lifecycle)

Although `operatorFee` is just one item in the six-way fee-split table (1%–3% share), at the ecosystem-narrative level it corresponds to an independent business role: the shelf's **"brand owner / operator"**. The protocol designs a full lifecycle around this role:

7.5.1 The Launcher Is the First Operator

- the caller of `initGood` **automatically becomes the good's `owner`**, written once with no extra authorization;
- that `owner` address is the default payee of `operatorFee` (auto-effective via the `commission[owner]` branch in `L_Good.allocateFee`);
- it also holds `modifyGoodByGoodOwner` (adjust user-section fee rates) and `lockGood` (emergency freeze) permissions.

7.5.2 Default 2-Year Operating Period (ecosystem-layer convention)

The protocol contract **itself does not hardcode "automatic switchover at end of operating period"** — the `owner` field is static storage and remains as-is unless `changeGoodOwner` is actively called. But at the ecosystem level, **2 years** is recommended as the default operating period for the first operator, because:

- similar to the primary market's "post-IPO lockup", it gives the launcher enough time to monetize early promotion investment;
- 2 years is also the typical cycle for most tokens to go from "launch" to "reaching stable depth";
- after it expires, market-based transfer via an "operating-rights auction" is recommended, letting the party most willing to invest in the shelf's next stage take over (see §7.5.3).

If governance wishes to put "automatic switchover at expiry" on-chain, a future contract upgrade could add an `ownerExpiresAt` field + an automatic `changeGoodOwner` auction flow after expiry; the current version operates via ecosystem convention + governance recommendation.

7.5.3 Operating-Rights Auction: 70 / 30 Split

After the first operating period expires or the original owner voluntarily exits, the shelf's operating rights transition to the next operator via an **auction**. The auction proceeds in two stages:

1. **Off-chain / dedicated-frontend auction**: the subject is the " `owner` identity" — it does not include any supply receipts (`proofs`) or TTS hashrate, both of which are decoupled from the `owner` field; optionally English / Dutch / sealed-bid, chosen by governance;
2. **On-chain handover**: the auction proceeds are split by the protocol's recommended **70 / 30**:
   - 70% to the original `owner` (rewarding early operating investment);
   - 30% to the community treasury (into `commission[address(0)]` , later withdrawn in batch by the Market Admin via `collectCommission` , with usage decided by a Class-III budget proposal);
   - after the split is confirmed, the **Market Manager** calls `changeGoodOwner(goodid, new owner)` to complete the identity migration.

Boundary constraints:

- `changeGoodOwner` is a Market Manager permission (§10.3), meaning the final landing of the auction result is **governance-constrained**; details of proceeds splitting / dispute period / buyer eligibility require an off-chain process + on-chain multisig review;
- the auction **only transfers the `owner` field**; any LP-held `S_ProofState` is unaffected, and those LPs can continue to divest and collect historical fees;
- 70 / 30 is a **protocol-layer recommended ratio**; the concrete implementation (escrow contract, deposit, pricing anchor, etc.) iterates with community governance. The first governance version may be carried by a standalone `OperatorAuction.sol` contract, or directly by the operations team + multisig acting on its behalf.

7.5.4 What the Contract Layer Can Do Today

The current v2.0.0 contract directly supports the **permissioned transfer of `changeGoodOwner`** :

Function	Who can call	Effect
`changeGoodOwner(goodid, newOwner)`	Market Manager	immediately migrate the shelf's `owner` field
`modifyGoodByGoodOwner`	current `owner`	adjust the user-adjustable section (power / fee rates / chips)
`lockGood`	`owner` or Market Manager	emergency freeze

Advanced "market-based operating rights" features such as operating-period timing, auction 70/30 splitting, and deposit escrow **belong to the ecosystem layer / next-generation governance contract** and do not block the deployment and use of the current contract.

8 Liquidity Mining (LP → TTS)

Each investment, when `isPromised()` is true, triggers `L_Proof.stake(TTS_CONTRACT, msg.sender, investvalue)` , effectively converting the LP shares into TTS hashrate.

8.1 Daily Emission Formula (measured against `_stakeFee`)

```
function _stakeFee() internal {
    if (stakestate.amount0() + 86400 < block.timestamp) {
        stakestate = toTTSwapUINT256(uint128(block.timestamp), stakestate.amount1());
        uint128 leftamount = 2e20 > totalSupply ? 2e20 - totalSupply : 0;
        uint128 mintamount = leftamount < 1e12 ? 1e12 : leftamount / 18250;
        poolstate = add(poolstate, toTTSwapUINT256(ttstokenconfig.getratio(mintamount), 0));
    }
}
```

Precise on-chain semantics (clarified in the current version relative to earlier doc wording):

- triggered once every 86,400 seconds (1 day);
- total releasable amount `leftamount = 2e20 - totalSupply` (TTS supply cap = 2e20 wei, i.e. 200 million TTS at precision 12);
- while below the cap, mint `leftamount / 18250` (roughly 50-year linear);
- **minimum protection**: if the computed result < `1e12` (i.e. 1 TTS), force `1e12` ;

- the actual injected amount is then distributed to the current chain's staking pool by `chainRatio / 10000`.

8.2 `stake / unstake` Core Semantics

- only contracts with `isCallMintTTS()` permission can call (= `TTSwap_Market` in a normal deployment);
- `stake(user, value)`: converts the user's proof value to `netconstruct` at the current `poolstate/stakestate` ratio, accumulating into `stakeproof[keccak256(user, market)]`;
- `unstake(user, value)`: converts the user's shares to TTS yield `_mint(user, profit)`, where profit = the user's share of the current poolstate − the already-accounted construct.

8.3 The LP's Three Stacked Yields

When `disinvestProof` is triggered, a single LP divestment **simultaneously settles three cash flows**, corresponding to three independent logic segments in the code:

Yield source	Code path	Denomination	Relationship to price
① Principal	`L_Good.disinvestGood` returns via `proofInvest1 × shares / proofShares0`	main Token (`currentgood`)	**independent of the current pool price**, strictly by proof historical quantity
② Trading-fee share	`L_Good.allocateFee` splits `liquidFee + customerFee` to `commission[sender]`	main Token	positively correlated with in-pool accumulated `fee`
③ TTS mining yield	`L_Proof.unstake` (when `disinvestTTSValue > 0`)	**TTS**	positively correlated with `poolstate` accumulated emission × the user's hashrate share

> The three are merged into a single transfer by the same `disinvestProof` call; the LP does not need to sign / collect separately. On partial divestment, all three are cashed out proportionally to the withdrawn shares, and the remainder continues mining.

Correspondence:

- principal protection corresponds to §5.4's "triple defense + zero impermanent loss";
- fee sharing corresponds to §7.2's six-way distribution (in most configs the LP takes the largest share of `liquidFee`, 50%–80%);
- TTS yield corresponds to the stake/unstake accounting of §8.1–8.2.

Compared to the Uniswap family with only "principal + IL correction" and Curve with only "principal + external CRV emission", TTSWAP's LP completes **base-currency compounding + six-way fee split + equity-type token** — three tiers of settlement — in a single divestment call, a triple-yield model natively supported by the protocol.

9 Tokenomics (TTS Tokenomics)

9.1 Basic Information (`TTSwap_Token.sol`)

Item	Value	Code location
Name / Symbol	TTSwap Token / **TTS**	constructor
Precision	**12 decimals**	`ERC20("TTSwap Token", "TTS", 12)`
Max supply	**200 million TTS** = `2e20` wei	`2e20` in `_stakeFee`
`left_share`	initial **4.5e19** (i.e. **45,000,000 TTS** at 12-decimal precision)	`uint128 public override left_share = 45_000_000_000_000_000_000;`
Public-sale hard cap	**250,000 USDT** = `2.5e11` (at 6-decimal precision)	`publicSell` reverts if `publicsell > 2.5e11`
Executor compensation	`executeFee = 5e10` = **0.05 USDT**	constant

Item	Value	Code location
Version	**1.16.0**	`string constant Version`
Cross-chain	`ttstokenconfig.ismain()` determines the main chain	`onlymain` modifier

9.2 Price-Doubling Unlock Mechanism (Fully On-Chain)

The on-chain code's implementation path is spelled out below:

Allocation phase (`addShare` / `permitShare`):

`s_share { leftamount, chips, metric }` :

- `leftamount` : remaining unlockable amount;
- `chips` : total number of shares (each unlock is 1/chips);
- `metric` : unlock-count counter (+1 after each `shareMint`).

Unlock phase (`shareMint`):

```
function shareMint() external onlymain {
    if (!I_TTSwap_Market(marketcontract).ishigher(
            address(this), usdt,
            2 ** shares[msg.sender].metric * 2 ** 128 + 20_000_000
    )) revert TTSwapError(68);
    if (shares[msg.sender].leftamount == 0) revert TTSwapError(69);
    uint128 mintamount = shares[msg.sender].leftamount / shares[msg.sender].chips;
    shares[msg.sender].leftamount -= mintamount;
    shares[msg.sender].metric += 1;
    _mint(msg.sender, mintamount);
}
```

Core on-chain semantics:

- before each `shareMint` , call `TTSwap_Market.ishigher(TTS, USDT, priceThreshold)` for a **price-trigger check**;
- the price threshold = $2^{\text{metric}} \times 2^{128} + 20{,}000{,}000$, i.e. **the threshold doubles per unlock**;
- the per-unlock amount = `leftamount / chips` (the larger `chips` , the smaller each unlock);
- **if the token price has not risen to the threshold → `ishigher` returns false → `shareMint` reverts**, and the unlock never happens.

The 4C role shares are implemented by giving different recipients different `leftamount / chips` ratios in `addShare` :

4C role	Total allocation share	`chips` reference (per-unlock cap)	Trigger example
Founder	20%	`chips = 20` → 1/20 (5%) per unlock	at 1,000× rise, cumulative unlock ≈ 37%
Partner	12%	`chips = 14` → 1/14 (7.1%) per unlock	at 400× rise, cumulative ≈ 45%
Value	44%	`chips = 12` → 1/12 (8.3%) per unlock	at 200× rise, cumulative ≈ 50%
Capital	24%	`chips = 8` → 1/8 (12.5%) per unlock	at 100× rise, cumulative ≈ 60%

9.3 `permitShare` Asynchronous Batch Allocation

The Token Admin can sign an `s_share` struct offline via EIP-712, with signature fields:

```
permitShare(
    uint128 amount,
    uint120 chips,
    uint8  metric,
    address owner,
    uint128 existamount,
    uint128 deadline,
    uint256 nonce
)
```

The recipient calls `permitShare(share, deadline, signature, signer)` to claim their share on-chain, suited for **large-scale airdrops / partner quotas / ecosystem incentives**: batched, asynchronous, and zero-gas signatures.

9.4 Public-Sale Tiered Pricing (on-chain hard cap)

`publicSell(usdtamount, data)` mints TTS by three tiered ratios:

Tier	Raised USDT range	Unit price	Contract constant
1	≤ 87,500	**1 USDT = 25 TTS**	`usdtamount × 25_000_000`
2	87,500 < x ≤ 162,500	**1 USDT = 20 TTS**	`usdtamount × 20_000_000`
3	162,500 < x ≤ 250,000	**1 USDT = 16 TTS**	`usdtamount × 16_000_000`

> Exceeding the 250,000 USDT hard cap reverts with `TTSwapError(70)`; the USDT is withdrawn to a designated address by the Token Admin via `withdrawPublicSell`.

9.5 Inflation and Deflation

Inflation (supply side):

$$\text{daily_mint} = \frac{2 \times 10^{20} - \text{totalSupply}}{18{,}250} \times \frac{\text{chainRatio}}{10{,}000}$$

roughly 50-year linear release, all distributed to LP mining.

Deflation (demand side):

- `commission[address(0)]` (protocol treasury) is collected in batch by the Market Admin via `collectCommission`;
- the community treasury performs **secondary-market buyback of TTS and burns it**, forming a deflationary engine;
- `burn(value)` for user self-burn + DAO Admin `burn(from, amount)` as backstop.

9.6 DAO Emergency Backstop

The contract reserves DAO Admin-exclusive functions:

- `mint(to, amount)`: contract-migration compensation / emergency repair (not bound by `left_share` and `publicSell` limits);
- `burn(from, amount)`: forced burn (contract migration / vulnerability backstop);

all operations emit `Transfer` events, on-chain auditable by the community.

10 Protocol Governance and Role System

10.1 `UserConfig` Bitmap (256-bit)

Bit	Field	Semantics
255	DAO Admin	highest authority, `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin / setEnv / disableUpgrade / mint / burn / fixLeftSh`

Bit	Field	Semantics
254	Token Admin	`setTokenManager / setCallMintTTS / setRatio / addShare / burnShare / withdrawPublicSell / permitShare (signer)`
253	Token Manager	`setBan`
252	Market Admin	`modifyGoodByAdmin / collectCommission(protocol)`
251	Market Manager	`modifyGoodByManager / changeGoodOwner / lockGood / freezeMarket`
250	CallMintTTS	`stake / unstake` permission (granted to the Market contract)
249	Stake Admin	`setStakeManager`
248	Stake Manager	liquidity-mining operations
160	Ban flag	banned address, automatically loses gate/referral eligibility on divestment
0–159	Referral address	the user's referrer address

Key permission chain: `DAOAdmin → {TokenAdmin, MarketAdmin, StakeAdmin} → corresponding Manager`.

10.2 `goodConfig` Three-Tier Modification Isolation (v2.0.0)

Function	Call permission	Modifiable section	Purpose
`modifyGoodByGoodOwner`	**Good Owner**	user section (bits 166–128: power / fee rates / chips)	issuer's daily tuning
`modifyGoodByManager`	**Market Manager**	admin section (bits 246–191: fee split / safeLine / isPromise, etc.)	admin intervention, validated by `checkGoodConfig`
`modifyGoodByAdmin`	**Market Admin**	super-admin section (bits 255–247: `isValueGood` / ERC type)	highest-level elevation

10.3 Token Management Tools

Function	Permission	Notes
`lockGood`	Token Owner or Market Manager	instantly freeze a good's trading (`isFreeze = true`)
`changeGoodOwner`	Market Manager	migrate a good's ownership
`cancelNonce`	any user	increment `nonces[_trader]` to invalidate unused signatures
`queryCommission`	any user (view)	batch-query multiple goods' accumulated commission for an address
`collectCommission`	any user + Market Admin (the latter can collect the protocol treasury)	batch-collect commission of multiple goods

10.4 Ecosystem Yield Roles

Role	Typical share	Mechanism
LP	50–80%	`liquidFee → commission[sender]`
Token Owner	1–3%	`operatorFee → commission[owner]`

Role	Typical share	Mechanism
Gate (frontend / aggregator)	5–25%	`gaterFee` → `commission[gate]`
Referral	5–10%	`referFee` → `commission[referral]`
Customer	-10% (discount rebate)	`customerFee` → `commission[sender]` or gate
Platform	2–8%	`platformFee` → `commission[address(0)]` (community treasury)
Executor	fixed `executeFee`	X402 gas-payment compensation

10.5 Separation-of-Powers Governance Architecture (Community / CEO / Board)

The `UserConfig` bitmap only defines the "keys" at the contract layer. At the governance-architecture level, these keys are mapped to three mutually independent roles, forming a triangular check-and-balance structure of "**community decision-making + CEO execution + board supervision**":

Governance role	Corresponding contract keys	How produced	Core responsibilities
Community (DAO)	no direct keys, calls via the governance-contract executor	any TTS holder can participate	elect the CEO, authorize board seats, pass all Class I–IV proposals
CEO (execution layer)	**Market Admin + Token Admin** (`UserConfig[252] / [254]`)	elected by a Class-I community proposal, removable at any time	form the R&D / marketing / operations teams, hold business keys, emergency response (`freezeMarket` / `freezeToken`)
Board (supervision layer)	**DAO Admin** (`UserConfig[255]`)	multisig formed by **Type-A partner TTS holders**	`disableUpgrade` / emergency `mint/burn` / appoint lower-tier Admins; off-chain responsible for supervision + arbitration + values review

Check-and-balance points:

1. **Separation of decision and execution power**: the CEO cannot unilaterally modify appointments, misappropriate the treasury, or trigger `disableUpgrade` ; all organizational personnel / protocol changes / budgets / major matters must go through community proposals;
2. **The board is independent of the CEO**: the board holds the DAO Admin key but **holds no Market/Token Admin key**; the CEO cannot appoint or dismiss directors; the board can initiate a **reconsideration motion** against proposals that violate the core values (zero impermanent loss, one-token-one-pool, six-way split summing to 100%, users' funds withdrawable, `disableUpgrade` irreversible), sending the original proposal into a 72h cooling-off period requiring a community re-vote;
3. **Separation of emergency button and permanent resolution**: `freezeMarket / freezeToken / lockGood` are in the CEO team's hands (second-level response, reversible); `disableUpgrade` is in the board's hands (permanently irreversible, requiring a Class-IV community proposal + Timelock authorization);
4. **External participants are not bound by governance**: the fund-withdrawal ability of suppliers, customers, referrers, and shelf owners is hardcoded by the contract; no governance resolution can block `disinvestProof` / `buyGood` / `collectCommission` .

Proposal classification (resolution level):

Class	Typical scenario	Recommended threshold	Timelock
I Organizational personnel	nominate / remove CEO, authorize Admin keys	simple majority	short (24–48h)
II Protocol change	`modifyGoodByAdmin` to elevate a value Token, `upgrade` to V2, adjust six-way split caps	simple majority	medium (48–72h)
III Budget & finance	use of `collectCommission(protocol)` , `withdrawPublicSell` withdrawal, `addShare/permitShare` batch allocation	simple majority	medium (48–72h)
IV Major matters	`disableUpgrade` , emergency `mint/burn` , governance-contract parameter change	supermajority (≥ 2/3)	long (≥ 7 days)

> **Implementation phases**: at the protocol's early launch it operates in a semi-on-chain mode of "off-chain multisig + on-chain hash attestation of key points"; mid-term introduces a governance contract + Timelock; long-term, an independent board-contract module carries the "reconsideration lock + automatic executor", and finally after `disableUpgrade` executes, the DAO Admin key permanently expires.

The contract-layer `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin` are the only on-chain entries to this triangular architecture: **every key is a "working key" dynamically added or removed by community proposals**, not any individual's private property.

11 Contract Upgrade and Emergency Circuit Breaker

TTSWAP adopts the **Delegatecall Proxy** pattern. In production deployment, users and integrators should interface with the **Proxy address** (`TTSwap_Market_Proxy` / `TTSwap_Token_Proxy`); the logic contracts (`TTSwap_Market` / `TTSwap_Token`) serve only as `implementation` pointer targets.

11.1 Proxy Architecture

Contract	Role	Key state
`TTSwap_Market_Proxy`	market-state container + forwarding	`implementation` , `TTS_CONTRACT` (immutable), `nonces` , `upgradeable`
`TTSwap_Market`	logic implementation	`goods` , `proofs` , `executeFee` ; retains `implementation` / `upgradeable` placeholder slots aligned with the Proxy
`TTSwap_Token_Proxy`	token-state container + forwarding	`balanceOf` , `allowance` , `userConfig` , `stakestate` , `left_share` , `implementation`
`TTSwap_Token`	logic implementation	`shares` , `stakeproof` , `marketcontract` , `poolstate`

Deployment order (recommended):

1. deploy `TTSwap_Token` logic → `TTSwap_Token_Proxy` (DAO Admin, `ttstokenconfig` , metadata);
2. deploy `TTSwap_Market` logic → `TTSwap_Market_Proxy` (bind `TTS_CONTRACT`);
3. DAO calls `setEnv(market_proxy)` ; the Token Admin grants the Market `setCallMintTTS` and the Market Admin/Manager roles.

11.2 Upgrade and Circuit Breaker

Function	Contract	Permission	Purpose
`upgrade(address newImpl)`	Market/Token Proxy	Market/Token Admin, and `upgradeable == true`	hot-upgrade the logic contract address
`disableUpgrade()`	Market Proxy	**DAO Admin**	permanently disable upgradability (`upgradeable = false`)
`freezeMarket()`	Market Proxy	Market Manager, and `upgradeable == true`	emergency breaker: `implementation = address(0)`
`freezeToken()`	Token Proxy	Token Manager, and `upgradeable == true`	emergency breaker for the token logic

> **Separation of powers**: the upgrade authority is with the Admin (day-to-day ops), the permanent-lock authority is with the DAO Admin (governance endgame), and the breaker authority is with the Manager (fast response). The Proxy `fallback` reverts with `TTSwapError(63)` when `implementation == address(0)` .

12 Main Functions and On-Chain Gas

> Gas data comes from the v2.0.0 snapshot baseline; the v2.0.0 swap engine switched to slice iteration, so actual gas may vary slightly and should be re-measured against `.forge-snapshots/` after deployment.

12.1 Contract Versions

Contract	Version	Deployment form
TTSwap_Market	**2.0.0**	Proxy + Logic
TTSwap_Token	**1.16.0**	Proxy + Logic
TTSwap_Market_Proxy / TTSwap_Token_Proxy	—	user entry addresses

12.2 Per-Call Gas

Function	Steady-state gas	Notes
`buyGood` ERC20 → ERC20	~65k	includes `buyGoodInput/Output` slices (v2 snapshot baseline)
`payGood` ERC20 cross-token	~70–90k	exact-out reverse path
`buyGood` NativeETH → ERC20	~72k	no ERC20 inbound transfer
`investGood` single-sided append	99–120k	auto-priced at the in-pool price
`disinvestProof`	128–184k	includes six-way distribution + conditional unstake
`initGood`	~400k	first pool creation + proof initialization (no stake)
`multicall` (2× buyGood)	~130k+	depends on the number of sub-calls
`goodWelfare`	~61k	
`collectCommission` (single good)	~25k	

12.3 Comparison with Mainstream Protocols (full swap tx)

Protocol	Full swap tx gas	Structural characteristics
TTSwap	**~93k**	Singleton + single call + no callback
Uniswap V4	130–180k	`unlock → unlockCallback → sync/settle/take` multi-stage
Uniswap V3	~127k	SwapRouter + NFPM
Uniswap V2	~102k	Router + Pair
Curve	115–135k	Vyper + iterative D
1inch Fusion+	200–400k	aggregation routing

Sources of structural savings: no `unlock/lock` shell (–5k–10k), no Router layer, no Permit2 double-signing, no `sync/settle/take` three-stage, and proof as a mapping rather than an ERC721 (saving ~50–80k NFT mint).

13 Typical Scenarios

Scenario	Core functions	Recommendation
Long-tail token launch (Launchpad / IDO)	`initGood` + `investGood` + `isPromised` + `refreshPromise`	★★★★★
Single-sided market making (zero IL)	`investGood` + `disinvestProof`	★★★★★
Cross-token swap (specify input)	`buyGood`	★★★★★
Cross-token payment (specify output)	`payGood`	★★★★★
Same-token X402 collection	`payGood` (same-good fast path)	★★★★★
Relayer gas payment	`buyGood` / `payGood` + EIP-712	★★★★

Scenario	Core functions	Recommendation
Batch orchestration (swap + collect commission)	`multicall`	★★★★
Value Token deployment	`initGood` + `modifyGoodByAdmin(isValueGood)`	★★★★★
B2B2C aggregated distribution	six-way fee split + `setReferral`	★★★★★
Tokenized equity / RWA listing (roadmap)	`T_GoodKey(ercType=2)` + `initGood` + `buyGood`	★★★★ (pending ERC-1155 branch)
Prediction-market outcome-share trading (roadmap)	`T_GoodKey(ercType=3)` + `initGood` + `buyGood`	★★★★ (pending ERC-6909 branch)
Stablecoin low-slippage swap	`buyGood` + larger M (`power`)	★★★
High-frequency arbitrage (aggregator integration)	`multicall` batched `buyGood` / `payGood` (mind the run-block slot)	★★★

14 Contact

- **Twitter**: @ttswapfinance
- **Telegram**: @ttswapfinance
- **Email**: bussiness@ttswap.io
- **Discord**: ttswap
- **Website**: ttswap.io
- **GitHub**: github.com/ttswap

15 Legal License

File category	License
`src/TTSwap_Market.sol` / `TTSwap_Market_Proxy.sol` / `TTSwap_Token.sol` / `TTSwap_Token_Proxy.sol`	BUSL-1.1 (Business Source License)
`src/libraries/L_Good.sol` / `L_Proof.sol`	BUSL-1.1
`src/base/ERC20.sol`	AGPL-3.0-only
`src/interfaces/*.sol`	MIT
`src/libraries/L_Currency.sol` / `L_Error.sol` / `L_GoodConfig.sol` / `L_UserConfig.sol` / `L_TTSTokenConfig.sol` / `L_TTSwapUINT256.sol` / `L_SignatureVerification.sol`	MIT
`src/libraries/L_Transient.sol`	GPL-3.0-or-later
`src/interfaces/IWETH9.sol`	GPL-2.0-or-later

Detailed copyright information is governed by the `LICENSE` file in the GitHub repository. The core business logic is protected under BUSL-1.1, while interfaces and utility libraries use the MIT license for ease of community integration.

Appendix A: Detailed Derivation of the Value-Conservation Trading Model's Core Formulas

TTSWAP's trading process proceeds in two steps. The swap kernel **hardcodes the dual coefficient as the constant 2**.

A.1 Step One: Input Side → Transferred Value ΔV

Instantaneous price:

- before trade: $P_{\text{before}} = V_A / Q_A$;
- after trade (assumed): $P_{\text{after}} = V_A / (Q_A + \Delta a)$.

The protocol computes the average unit price with a harmonic-mean "execution density" (Δa being the net input after deducting `sellFee` ; the code iterates in slices of `current_quantity / 100` , see §3.1):

$$\Delta V = \frac{2 \cdot V_A \cdot \Delta a}{2 \cdot Q_A + \Delta a}$$

A.2 Step Two: Output Side → Token Quantity to Release Δb

The output side maps the transferred value back to a quantity with **the same constant 2**, with no dual transform and no `getK2` :

$$\Delta b = \frac{2 \cdot Q_B \cdot \Delta V}{2 \cdot V_B + \Delta V}$$

The code accumulates Δq_i in slices of `current_value / 100` , and after the loop deducts the output-side fee via `getBuyFee` (`L_Good.buyGoodOutput`).

A.3 TTSwap's Tunable Dimensions

- **M (liquidity amplification, `getPower / getLimitPower`)**: amplifies pool depth via the virtual amount; for stablecoins / highly correlated assets, a larger M flattens the balance-zone curve, approaching constant-sum.
- **`safeLine ( getSafeLineUpper/Lower )`**: constrains a single pool's Q tradeable range, playing the role of "add friction / block on deviation"; crossing the bound reverts with `TTSwapError(55/56)` .
- **Fee rates (buy / sell / invest / disinvest)**: independently configurable per token at 1/10000 precision.
- **O(1) algebra**: only add/subtract/multiply/divide throughout, with no exponentiation, no iteration, and no external library calls.

A.4 Connection with CPMM

The formula is a standard harmonic mean:

$$\Delta V = \frac{2V_A \Delta a}{2Q_A + \Delta a}, \quad \Delta b = \frac{2Q_B \Delta V}{2V_B + \Delta V}$$

when $V_A = V_B$, it degenerates to $x \cdot y = k$ (see the CPMM equivalence in Appendix B).

Appendix B: CPMM Equivalence Proof ($V_A = V_B$)

Let $V_A = V_B = V$ and $2 = K_B = 2$. Substituting into the degenerate formula and combining:

$$\Delta V = \frac{2V \Delta a}{2Q_A + \Delta a}, \quad \Delta b = \frac{2Q_B \Delta V}{2V + \Delta V}$$

Substituting and simplifying (divide numerator and denominator by $2V$, arrange the denominator as $\frac{2(Q_A+\Delta a)}{2Q_A+\Delta a}$):

$$\Delta b = \frac{Q_B \cdot \Delta a}{Q_A + \Delta a}$$

i.e. Uniswap V2's standard output formula $\Delta y = \frac{y \Delta x}{x + \Delta x}$. **QED**.

This shows that TTSWAP, when $V_A = V_B$, has the same market depth and pricing behavior as Uniswap; while $V_A \neq V_B$ naturally extends to a weighted pool (Appendix C).

Appendix C: Balancer Equivalence Proof ($V_A \neq V_B$)

C.1 Balancer Recap

Weighted constant product: $(Q_A)^{W_A} \cdot (Q_B)^{W_B} = k$, spot price:

$$SP_{\text{Balancer}} = \frac{Q_A \cdot W_B}{Q_B \cdot W_A}$$

C.2 TTSWAP Marginal Price

Taking the limit $\Delta a \to 0$ of the input-side formula (coefficient always 2):

$$dV = \frac{V_A}{Q_A} da, \quad db = \frac{Q_B}{V_B} dV$$

Combining:

$$SP_{\text{TTSWAP}} = \frac{da}{db} = \frac{Q_A \cdot V_B}{Q_B \cdot V_A}$$

C.3 Comparison Conclusion

Mapping $V \leftrightarrow W$, the two spot prices are **exactly identical**. Under large trades, TTSWAP's harmonic-mean curve closely fits Balancer's exponential curve in convexity, asymptotes, and slippage direction, but TTSWAP **requires no exponentiation** and its gas consumption is $O(1)$ constant.

Appendix D: Curve StableSwap Equivalence Proof

D.1 Curve Recap

Constant-sum + constant-product combination:

$$A \cdot n^n \cdot \sum x_i + D = ADn^n + \frac{D^{n+1}}{n^n \prod x_i}$$

The amplification coefficient A makes stablecoin-zone slippage < 1 bp. But solving D requires Newton-Raphson iteration.

D.2 TTSWAP's Two-Parameter Balance-State Approximation

StableSwap-style low slippage is mainly achieved by **M (liquidity amplification)**, while deviation regression is borne by the **`safeLine` boundary**:

- **M (liquidity amplification, `getPower`)**: increasing M for stablecoins makes the balance-zone curve near-straight (constant-sum);
- **`safeLine` (`getSafeLineUpper/Lower`)**: monitors how far Q deviates from the base inventory `I + virtual`; crossing the bound reverts (`TTSwapError(55/56)`), forcing subsequent trades to pull Q back in reverse.

Stablecoin balance-state math: when $V_A = V_B = V$, $Q_A = Q_B = Q$, and M is large (in-pool virtual quantity dominates):

$$\Delta V \approx \frac{V_A \Delta a}{Q_A}, \quad \Delta b \approx \frac{Q_B \Delta V}{V_B} = \Delta a$$

i.e. $\Delta a \approx \Delta b$, a 1:1 swap with minimal slippage.

Deviation-state regression: when a trade pushes Q off balance ($Q \ll I$ or $Q \gg I$), the harmonic-mean curve's own slippage amplifies non-linearly with deviation, and layered with the `safeLine` upper/lower bounds directly reverting on touch (`TTSwapError(55/56)`), together pushing subsequent trades to pull Q back near `I + virtual`. This mechanism of "**auto-friction on deviation / block at the boundary**" is equivalent to Curve's soft transition from "constant-sum dominant" to "constant-product dominant" when $x_i \neq x_j$.

D.3 Engineering Advantages

Dimension	Curve	TTSWAP
Computational complexity	iterate to solve D + high powers	add/subtract/multiply/divide + O(1)
Gas cost	higher	extremely low (92,747 steady-state)
Applicable range	stablecoin pairs	general tokens, M and fee rates tunable (K fixed at 2)
Parameter flexibility	slow governance adjustment of A	per-token independent config of M, `safeLine` , fee rates
Deviation-state response	A fixed, curve auto-transitions toward product	natural curve slippage + `safeLine` boundary blocking drives Q regression
Balance-state low slippage	increase A	increase M (orthogonal dimension)

Appendix E: `goodConfig` 256-bit Layout (Complete)

```
Bit 255           isvaluegood         1 bit
Bits 254–247      erctype             8 bits
Bit 246           isFreeze            1 bit
Bit 245           isVerified          1 bit
Bit 244           isPromise           1 bit
Bits 243–241      liquidFee           3 bits   × 0.1   default:6
Bits 240–237      operatorFee         4 bits   × 0.02  default:1
Bits 236–234      gateFee             3 bits   × 0.04  default:5
Bits 233–229      referFee            5 bits   × 0.01  default:8
Bits 228–224      customerFee         5 bits   × 0.01  defualt:8
Bits 223–219      platformFee         5 bits   × 0.01  default:2
Bits 218–214      limitPower          5 bits   × 1
Bits 213–204      safeLine            10 bits
Bits 203–192      contractType        12 bits
Bit 191           reversed            1 bit
Bits 190–179      getLastRun          12 bits
Bits 178–167      reversed           12 bits  × 10
Bits 166–162      power               5 bits   × 1
Bits 161–154      disinvestChips      8 bits   x 4
Bits 153–148      investFee           6 bits   × 0.0001  defualt:0.08%
Bits 147–142      disinvestFee        6 bits   × 0.0001  defualt:0.08%
Bits 141–135      buyFee              7 bits   × 0.0001  defualt:0.08%
Bits 134–128      sellFee             7 bits   × 0.0001  defualt:0.08%
Bits 127–0        marketValue field   128 bits
```

Six-way fee-split sum constraint: `checkGoodConfig` enforces `liquidFee + operatorFee + gateFee + referFee + customerFee + platformFee = 100%` (in units of 10% / 2% / 4% / 1% / 1% / 1%).

Precision-check functions:

- `getPower(config)`： `config[166..162] × 100` , default 100 (i.e. 1×);
- `getLimitPower(config)`： `config[218..214] × 100` , the investment leverage cap;
- `getSafeLineUpper/Lower(config, amount)` : bits 213–204, default 80;
- `getInvestFee / getDisinvestFee / getBuyFee / getSellFee` : 1/10000 precision, all default to 8 (0.08%).

Appendix F: Full Invest and Divest Calculation Flow (code-aligned)

F.1 Initial State

Item	Value
$V =$ investState.amount1	40,000
$Q =$ currentState.amount1	40,000
$I =$ currentState.amount0	40,000
$S =$ investState.amount0	40,000
$M =$ getPower	5

F.2 Invest 200 Units (`investGood` core flow)

1. `enpower = getInvestPower(goodid)` (dynamically scaled by Q/I, capped by `getPower()`);
2. `investFeeQuantity = getInvestFee(200)` (assumed = 0 in this example);
3. `investQuantity = (200 − 0) × enpower / 100` (virtual quantity);
4. `investValue = poolPrice × investQuantity` , truncated by `getInvestThreshold` ;
5. `investShare = goodShares / goodInvestQuantity × 200` ;
6. update the pool: `currentState` , `investState` , `goodConfig.amount1` (virtual amount);
7. update the proof: shares / state / invest;
8. `L_Proof.stake(TTS, msg.sender, investvalue_actual)` issues TTS hashrate.

F.3 After Market Changes (assuming the pool gains yield)

- V = 41,000 (unchanged)
- Q = 41,000 → **44,000**
- I = 40,200 → **43,000**
- S = 40,200

F.4 Divest 100 Shares (`disinvestProof` core flow)

1. identity check: `S_ProofKey(_trader, currentgood).toId() == _proofid` ;
2. compute the return by proof historical ratio:
   - `actualDisinvestQuantity = proofInvest1 × goodshares / proofShares0 = 200 × 100 / 200 = 100` ;
   - `virtualDisinvestQuantity = proofInvest0 × 100 / 200 = 500` ;
3. compute the current value from the current pool:
   - `profit = currentState.amount0 / investState.amount0 × shares = 43000/41000 × 100 ≈ 104.88` ;
4. divestment fee: `actual_fee = getDisinvestFee(virtualDisinvestQuantity)` (assumed 0.5%, = 2.5);
5. pool update:
   - `currentState -= (profit, virtualDisinvestQuantity + profit − actualDisinvestQuantity)` ;
   - `investState -= (shares, disinvestvalue.amount0)` ;
   - `currentState += (actual_fee, actual_fee)` (divestment fee back to pool);
6. net profit = `profit − actualDisinvestQuantity = 104.88 − 100 = 4.88` ;
7. `allocateFee(profit=4.88, gate, referral, quantity=100, sender)` six-way distribution;
8. `L_Proof.unstake(TTS, sender, divestvalue.amount1())` .

F.5 Disinvest Chips Rate-Limiting

If `virtualDisinvestQuantity > getDisinvestChips(currentState.amount1)` → revert `TTSwapError(27)` ; if `disinvestvalue.amount0 > getDisinvestChips(investState.amount1) || disinvestvalue.amount0 < 10000` → revert `TTSwapError(26)` .

Appendix G: Full Token Swap Calculation Flow

G.1 Initial State

- Token A: V=40000, Q=20000, price=2.0, fee=0.01%;
- Token B: V=20000, Q=40000, price=0.5, fee=0.01%;

G.2 User Inputs 2500 Token A (`buyGood` ERC20→ERC20)

1. `sellFee = getSellFee(2500)` ;
2. `Δa (net) = 2500 − sellFee` ;
3. Step 1 (`buyGoodInput` , slice iteration):
   each slice $\Delta V_i = \frac{2V\Delta q_i}{2Q + \Delta q_i}$, accumulated into the total ΔV;
4. Step 2 (`buyGoodOutput` , slice iteration):
   each slice $\Delta q_i = \frac{2Q\Delta V_i}{2V + \Delta V_i}$, accumulated into the total output;
5. output fee `buyFee` , user net receipt = gross output − `buyFee` .

G.3 State Update

- Token A: `currentState` amount0 += 0.25, amount1 += 2500
- Token B: `currentState` amount0 += 0.84, amount1 -= (8420.3 - 0.84)
- `investState` all unchanged

G.4 Price Change

- Token A new price ≈ 40000 / 22500 ≈ 1.78 (down)
- Token B new price ≈ 20000 / 31580 ≈ 0.63 (up)

Appendix H: Fee Calculation and Allocation

H.1 Accumulation Formula

Each swap:

$$I_{\text{new}} = I_{\text{old}} + \text{fee}$$

Net value:

$$\text{NetValue} = \frac{I + \text{AccumulatedFees}}{S}$$

As trading volume grows, the net value increases monotonically.

H.2 `allocateFee` Distribution Pseudocode

```
platformFee = getPlatformFee128(profit);
profit -= platformFee;

liquidFee   = getLiquidFee(profit);
operatorFee = getOperatorFee(profit);
gateFee     = getGateFee(profit);
referFee    = getReferFee(profit);
customerFee = getCustomerFee(profit);

if (referral == 0) {
    if (gate == 0) {
        commission[0]       += (profit - liquidFee + platformFee);
        commission[sender]  += (liquidFee + actualDivestQuantity);
    } else {
        commission[sender]  += (liquidFee + actualDivestQuantity);
        commission[gate]    += (operatorFee + customerFee);
        commission[0]       += (profit + platformFee - liquidFee - operatorFee - customerFee);
    }
} else {
    if (owner != 0) commission[owner]  += operatorFee;
    else            platformFee        += operatorFee;
    if (gate  != 0) commission[gate]   += gateFee;
    else            platformFee        += gateFee;
    commission[referral] += referFee;
    commission[0]        += platformFee;
    commission[sender]   += (liquidFee + customerFee + actualDivestQuantity);
}
```

H.3 Commission Collection 1-unit Sentinel

The contract reserves the lowest unit of 1 to avoid a cold SSTORE:

```
if (tranferamount > 1) {
    goods[normalgood].commission[msg.sender] = 1;
    normalgood.safeTransfer(msg.sender, tranferamount - 1);
}
```

Appendix I: Reverse-Swap Closure

Given V_A, Q_A, V_B, Q_B, the user inputs Δa to swap out Δb:

1. forward: $\Delta V = \frac{2V_A\Delta a}{2Q_A+\Delta a}$, $\Delta b = \frac{2Q_B\Delta V}{2V_B+\Delta V}$;
2. reverse input Δb:

$$\Delta V' = \frac{2V_B\Delta b}{2(Q_B-\Delta b)+\Delta b} = \frac{2V_B\Delta b}{2Q_B-\Delta b}$$

3. transforming the forward Δb formula: $\Delta V = \frac{2V_B\Delta b}{2Q_B-\Delta b}$, we get $\Delta V' = \Delta V$ (**value conservation**);
4. reverse output $\Delta a_{\text{out}} = \frac{2(Q_A+\Delta a)\Delta V}{2V_A+\Delta V}$;
5. substituting $\Delta V(2Q_A+\Delta a) = 2V_A\Delta a$ and simplifying: $\Delta a_{\text{out}} = \Delta a$.

QED. Path-independent, with no risk-free arbitrage.

Appendix J: Split-Trade Error

Definition: $\Delta a = x_1 + x_2$, comparing once vs twice.

Error formula (ignoring fees):

$$\text{Diff} = \frac{C_1(x_1+x_2)(V_A^2 - V_B^2)x_1x_2}{(C_2 + N(x_1+x_2))(C_2+Nx_1)(C_2+2V_Bx_1+Nx_2)}$$

where $C_1 = 2Q_BV_A$, $C_2 = 2V_BQ_A$, $N = V_A + V_B$.

J.1 Path-Independence Conditions

- $V_A = V_B$: Diff = 0 (strictly path-independent);
- $V_A > V_B$: Diff > 0 (a single trade is better);
- $V_A < V_B$: Diff < 0 (splitting the trade is better).

J.2 Error Magnitude

$$\frac{\text{Diff}}{\Delta b} \approx \left(\frac{\Delta a}{Q_A}\right)^2 \cdot \frac{V_A^2 - V_B^2}{V_B^2}$$

when a single $\Delta a/Q_A \approx 10^{-3}$, the relative error $\sim 10^{-6}$, negligible in engineering practice.

Appendix K: Triangular Closure Proof

The three-hop cycle $A \to B \to C \to A$:

K.1 Value Lemma

Each hop's ΔV is strictly equal to the previous hop's ΔV (from the invertible transform of the Δb formula):

$$\Delta V_{AB} = \Delta V_{BC} = \Delta V_{CA} = \Delta V$$

K.2 State Restoration

- Token B pool: $Q_B + \Delta b$ then out Δb, back to Q_B;
- Token C pool: $Q_C - \Delta c$ then in Δc, back to Q_C;
- Token A pool: $Q_A + \Delta a$ then out Δa_{final}.

K.3 Proof that $\Delta a_{\text{final}} = \Delta a$

$$\Delta a_{\text{final}} = \frac{2(Q_A + \Delta a)\Delta V}{2V_A + \Delta V}$$

substituting $\Delta V(2Q_A + \Delta a) = 2V_A\Delta a$ and expanding/simplifying, the equality holds. **QED**.

The Token A pool state is also restored, the system rolls back, and there is no risk-free arbitrage space.

End of Whitepaper · TTSWAP · 2026-05-09